AMD Developer Hackathon: ACT III | Track 2: Health and Wellbeing

Multi-Agent AI Care Coordinator

Project document pack: five separate documents, kept intact and in order

Team: Ishaan, Mariam, Mozzam / Askeri

Contents

Part 1: Product Requirements Document (PRD) pages 2-12

Full product spec: hackathon tracks and Track 2 fit, goals, users, agents, functional and safety requirements, evaluation, team plan, submission checklist, risks.

Part 2: Project Workflow pages 13-18

End-to-end workflow: intake and safety screen, care contract, four Workers, verification and retry loop, human approval, live dashboard, evaluation and demo.

Part 3: Tech Stack and System Blueprint pages 19-21

Implementation map: specified vs proposed components (GPU and serving, orchestration, ingestion, medication data, dashboard, local data), safety and evaluation.

Part 4: Workflow Flowchart pages 22-24

Visual flowchart of the full process from patient request to the live dashboard, shown on two pages (top half, then bottom half).

Part 5: CarePilot AI Features pages 25-29

Project concept and system features: cancer-patient record organization and doctor-visit preparation, ten core features, workflow, role of Python, technology fit, safety and limitations.

Each part starts with its own divider page. Content inside each part is unchanged from the original files. Part 5 is new in this version and is reproduced from the CarePilot AI project concept document.

PART 1

Product Requirements Document (PRD)

Full product spec: hackathon tracks and Track 2 fit, goals, users, agents, functional and safety requirements, evaluation, team plan, submission checklist, risks.

Pages 2-12 of 29

Product Requirements Document

Multi-Agent AI Care Coordinator

Private, source-checked care summaries from messy medical documents, running fully on AMD Instinct MI300X

This idea is good for Track 2 (Healthcare).
It matches the Track 2 challenge, Health and Wellbeing, on use case, demo and safety rules (see Section 3).

Event	AMD Developer Hackathon: ACT III - Track 2: Health and Wellbeing
Team (3 members)	Ishaan, Mariam, Mozzam / Askeri
Timeline	5-day build
Stack	AMD Instinct MI300X (192 GB), ROCm, vLLM, one open model, small safety classifier, amd-smi
Data	Synthetic patient records only
Status	Draft v1.1 for team alignment

1. Summary

Patients and caregivers receive discharge summaries, lab reports, prescriptions, specialist notes and imaging reports, but have no practical way to organize them. This product turns a plain-language request, such as *"help me prepare for my cardiology appointment tomorrow"* or *"explain these discharge instructions in simple words"*, into a clear, source-checked, patient-ready care summary.

Six cooperating agents share one open model served by a single vLLM instance on one MI300X. Every claim in the output is traced to the exact source line or removed. Uncertain items are flagged for a clinician. Emergencies trigger a hard stop. Nothing is final until the person explicitly approves it. No patient data leaves the machine, so the same stack can be deployed on a hospital's own servers.

2. AMD Developer Hackathon: ACT III - The Tracks

ACT III is a hybrid hackathon run by AMD and lablab.ai. The online build phase runs 12-18 October 2026, with an optional on-site phase on 17-18 October in Rome, Milan and Imperia (Italy). Prize pool is listed as $12,000+. Teams have 1-6 people. Every project must run a meaningful part of its workload on AMD infrastructure (AMD

Developer Cloud, Instinct accelerators, ROCm, Ryzen AI or Radeon), and that AMD part must be in the working product shown to judges. Participants choose **one challenge track** and build a working product for a real user.

Track	Theme	What to build (idea examples)	What good projects show
Track 1 Intelligent Industry	Help factories spot problems earlier and operate better	Predict equipment failures, detect defects from images or video, explain production anomalies, help technicians search manuals and maintenance records, recommend maintenance actions, monitor safety, reduce energy or material waste	A real manufacturing problem; a working product, not only a dashboard; results operators understand; measurable gain in time, cost, safety, quality or efficiency
Track 2 Health and Wellbeing **(OUR TRACK)**	Make a part of the patient journey safer, clearer or easier	Patient intake, medical record summaries, appointment preparation, care navigation, clearer discharge instructions, multilingual patient communication, healthcare document processing, admin workflow automation, wellbeing and prevention support	A clearly defined patient or staff need; responsible handling of health information; useful, understandable results; measurable gain in time, accuracy, accessibility or workload. Must show uncertainty and know when a human reviews or takes over
Track 3 Reinvent Commerce	Help a business understand its customers and take better action	Shopping assistants, product discovery and recommendations, customer support, demand and inventory planning, catalogue enrichment, review and sentiment analysis, promotion planning, churn prevention, in-store computer vision. Forecasts must not rely on a language model alone	A real customer or merchant problem; a useful commercial action; relevant personalization; measurable effect on revenue, margin, conversion, retention, inventory or service
Track 4 Create a New Kind of Experience	Build a media, marketing or entertainment experience not practical before generative AI	Personalized video, interactive stories, audience-driven entertainment, adaptive campaigns, generative brand experiences, responsive sound or visuals, accessible media. Demo must change for at least two audiences or inputs	An original concept; a strong user experience; human creativity combined with technology; realistic cost per generated experience

Note on the number of tracks: the official event page lists **four** challenge tracks (above), not three, plus one optional partner track from Evolus (below). Please re-check the page at kickoff, because organizers may change tracks, rules or prizes.

Partner track: Evolus Business Agents on AMD

Build an AI agent that runs a real business process end to end on the Evolus platform, powered by an open model served on AMD through your own vLLM endpoint on AMD Developer Cloud. It requires at least two Evolus building blocks (workflows, document extractors, AI Tools, knowledge libraries, MCP connectors, sub-agents, chat widget) and driving Evolus through its APIs. This is optional and separate from our Track 2 plan.

Source: lablab.ai event page for AMD Developer Hackathon: ACT III.

3. Why This Idea Fits Track 2 (Healthcare)

> **This idea is good for Track 2 (Healthcare).**

Track 2 expectation	How our product meets it
Handle long documents, scattered information and limited time	Document Classifier indexes the whole upload; Workers process in parallel; goal is under 2 minutes of preparation
Medical record summaries, appointment preparation, clearer discharge instructions, document processing	These are our core use cases: request such as 'prepare for my cardiology appointment' or 'explain these discharge instructions'
Show a realistic case from first input to final result	Demo uses a synthetic discharge summary, two lab reports and a prescription list, from upload to approved summary
Make uncertainty visible	'Cannot confirm' medication results, confidence level per section, and a 'needs clinician review' section
Know when a person must review or take over	Hard stop on emergencies; low-confidence hand-off to a human after two failed retries; explicit user approval before anything is final
Must not present autonomous diagnosis or treatment as medical advice	Fixed output structure, no diagnosis or treatment language, persistent disclaimer
Explain handling of incorrect or unsupported responses	Claim-to-source checking by the Manager/Compiler; untraceable claims removed or flagged
Explain handling of sensitive information, uncertain results, high-risk decisions, human review	All inference local on MI300X (no third-party AI API); synthetic data only; two-path high-risk handling; human review loop
Measurable improvement in time, accuracy, accessibility or workload	20-document test vs single-prompt baseline: unsupported-claim rate, reading level, time, planted errors caught
Meaningful AMD component in the working product	Entire pipeline runs on MI300X via ROCm and vLLM, with live amd-smi metrics in the dashboard

4. Problem and Opportunity

Care information is large, unstructured and scattered across document types. Caregivers under time pressure struggle to know what matters, what to ask the doctor, and whether medicines are safe together. General-purpose AI tools can produce fluent but unsupported claims and typically send data to third-party APIs, which is unacceptable for health data.

Opportunity: a local, multi-agent, verification-first pipeline that produces consistent output, shows its sources, admits uncertainty, and keeps a human in control.

5. Goals and Non-Goals

Goals

- Convert a plain-language request plus an uploaded document set into one structured, plain-language care summary.
- Trace every claim to a source line; remove or flag anything untraceable.
- Run 100% of AI inference locally on AMD MI300X via ROCm and vLLM, with no third-party AI API.
- Detect and hard-stop on current emergencies stated in the user's own request.
- Show all six agents sharing one GPU in a live dashboard with real hardware metrics.
- Measure real improvement against a single-prompt baseline and report only measured numbers.

Non-Goals

- No diagnosis, treatment recommendations or medical advice of any kind.
- No real patient data; no EHR/hospital system integration in this build.
- No multilingual translation, no mobile app, no user accounts or multi-tenant security.
- Not a replacement for a clinician or pharmacist; the keyword safety net is conservative, not complete.

6. Users

Persona	Need	How the product helps
Caregiver (primary)	Manage a relative's discharge papers, meds and appointments with little medical background	Upload documents, get a summary, doctor questions and medication checklist in about 8th-grade language
Patient	Understand own results and instructions before a visit	Plain-language explanation with sources and a clear list of what to confirm
Clinician / pharmacist (secondary)	Quickly see what the system could not verify	Dedicated 'needs clinician review' section with flagged items
Hackathon judge	See a safe, real, measurable system	Live dashboard, planted-error demo, baseline comparison

7. Standard Output Structure

Every output uses one fixed structure so results are consistent and no section adds unsupported claims. Each section carries a confidence level, and each claim links to its source line.

Section	Contents
1. Plain-language summary	Key clinical facts from the relevant documents, reading level about 8th grade
2. Questions for the doctor	Prioritized list tied to the request (for example, appointment prep)
3. Medication and allergy checklist	Each medicine checked against a trusted drug data source; 'cannot confirm' when no match
4. Needs clinician review	Uncertain items, unconfirmed interactions, and history-only risks (such as pregnancy or a past cardiac event)

8. System Architecture

The Main Agent plans and writes a short **care contract** (required information, which Worker handles each part, exact output format). Four Workers run in parallel. The Manager/Compiler merges and verifies. All agents call one vLLM instance; a small dedicated safety classifier runs on the same GPU. The 192 GB of MI300X memory holds the large model, the classifier and the KV cache for many concurrent requests, so there is no model swapping.

Agent	Role	Key outputs
Main Agent	Understands intent, splits work, writes the care contract, routes fixes and change requests, enforces the 2-retry limit	Care contract; routing decisions

Agent	Role	Key outputs
Intake Agent (Worker)	Interprets the request and identifies which documents are relevant	Intent and relevant-document list
Document Classifier Agent (Worker)	Sorts and indexes the full upload by type (labs, prescriptions, discharge notes, imaging) so nothing is missed	Document index grouped by type
Record Summarizer (Worker)	Extracts clinical facts from each relevant document	Fact list with source references
Medication Checker (Worker)	Compares every medicine to a trusted drug data source; reports 'cannot confirm' on no match; flags unconfirmed interactions	Medication checklist and flags
Manager/Compiler Agent	Merges the four outputs, verifies every claim against source, assigns confidence, then chats with the user for approve / improve / reject	Final summary; user dialogue

Request flow

- **1.** User enters request and uploads documents. Emergency check runs first (see Section 10).
- **2.** Main Agent writes the care contract and dispatches Workers in parallel.
- **3.** Manager/Compiler merges outputs and verifies each claim against its source line.
- **4.** On failure, Main Agent identifies the responsible Worker and returns the problem for a fix. Maximum two retries; then the item is marked low-confidence and handed to a human reviewer, never shown as advice.
- **5.** Manager/Compiler presents the result in chat with Approve / Improve / Reject. Change requests (for example, 'add a question about my allergy history') are routed by the Main Agent to the right Worker and re-verified.
- **6.** Nothing is marked final until the person explicitly approves.

9. Functional Requirements

Priority: **P0** = required for demo, **P1** = should have, **P2** = if time allows.

ID	Requirement	Priority
FR-1	Accept a plain-language request and a multi-file document upload (synthetic PDFs/text)	P0
FR-2	Main Agent produces a care contract listing needed info, Worker assignments and exact output format	P0
FR-3	Document Classifier groups and indexes every uploaded page by document type	P0
FR-4	Intake Agent selects relevant documents for the request	P0
FR-5	Record Summarizer extracts facts, each tagged with document and line reference	P0
FR-6	Medication Checker checks each medicine against a trusted drug data source and returns 'cannot confirm' when there is no match	P0
FR-7	Manager/Compiler merges outputs into the fixed four-section structure	P0
FR-8	Manager/Compiler links each claim to its exact source line and removes or flags untraceable claims	P0
FR-9	Each section carries a confidence level (high / medium / low)	P0

ID	Requirement	Priority
FR-10	Failed checks return to the responsible Worker via the Main Agent; max two retries, then low-confidence and human review	P0
FR-11	Chat interface presents results with Approve / Improve / Reject; improvements are routed and re-verified	P0
FR-12	Output is not final until explicit user approval	P0
FR-13	Every unconfirmed medication interaction is flagged with a 'please confirm this with your doctor' note	P0
FR-14	Each agent emits simple live events (start, finish, flag, retry) consumed by the dashboard	P0
FR-15	History-only risk items are flagged in 'needs clinician review' without stopping the run	P0
FR-16	Export of approved summary (PDF or text)	P2

10. Safety Requirements

Safety is a product feature, not an add-on. Two distinct situations are handled differently.

Situation	Behavior
Current emergency in the user's own request (for example chest pain, trouble breathing, overdose, heavy bleeding, thoughts of self-harm)	**Hard stop.** No summary is produced. The user is told to contact a clinician or their local emergency number.
Risky item only in document history (for example pregnancy, past cardiac event)	**No stop.** The item is flagged in the 'needs clinician review' section.
Claim cannot be traced to source	Removed or flagged; never presented as fact.
Medication not found or interaction unconfirmable	Reported as 'cannot confirm' and flagged for pharmacist or doctor.
Verification fails after 2 retries	Marked low-confidence and handed to a human reviewer.

- **SR-1** Emergency detection combines a fixed trigger list kept in code with a small model-based safety classifier on the same MI300X. Either can trigger the hard stop.
- **SR-2** The product states openly that keyword rules are a conservative safety net and can miss unusual phrasing.
- **SR-3** The system never presents diagnosis or treatment as medical advice, and shows this disclaimer in the UI.
- **SR-4** Only synthetic patient records are used anywhere in development, demo and evaluation.

11. Non-Functional Requirements

Area	Requirement
Privacy	All inference local on MI300X via ROCm and vLLM; no calls to third-party AI APIs; deployable on hospital-owned servers
Performance	Time to produce an approved-ready summary under 2 minutes for the demo document set; agents run concurrently on one vLLM instance without model swapping
Readability	Output at about 8th grade reading level

Area	Requirement
Accuracy	Unsupported-claim rate under 5% on the evaluation set
Consistency	Fixed output structure on every run
Transparency	Every claim source-linked; every uncertainty visible; confidence per section
Observability	Live agent events plus GPU utilization, tokens per second, latency and memory read from amd-smi
Portability	Open software stack; no proprietary model dependency

12. Care Dashboard

A real-time dashboard driven by agent events and amd-smi, designed so judges can see all six agents sharing one GPU.

Panel	Shows
Agent activity	Which of the six agents are active, idle, retrying or finished
Uncertainty flags	Where uncertainty was flagged and which section it affects
Hardware metrics	GPU utilization, tokens per second, latency, memory use (live from amd-smi on the MI300X)
Chat panel	Summary preview and Approve / Improve / Reject conversation with the Manager/Compiler

13. Evaluation Plan and Success Metrics

We run **20 synthetic documents** through (a) a plain single-prompt baseline and (b) our full pipeline, and compare results. We will report the numbers we actually measure, whether or not they meet the targets.

Metric	Method	Target
Unsupported-claim rate	Fraction of output claims not traceable to source text	Under 5%
Reading level	Standard readability score on the plain-language summary	About 8th grade
Time to produce	Pipeline wall-clock vs a manually timed baseline	Under 2 minutes
Planted errors caught	Count of deliberately planted errors detected, baseline vs pipeline	Pipeline catches more than baseline
Emergency recall (extra)	Share of scripted emergency phrasings that trigger the hard stop; also report misses	Reported honestly

14. Demo Scenario

- A caregiver uploads synthetic discharge documents: one discharge summary, two lab reports and a prescription list.
- The Document Classifier sorts and indexes them; Workers process in parallel while the dashboard shows all six agents on one GPU.
- The system produces the summary, doctor questions and medication checklist.
- One **planted medication interaction cannot be confirmed**; the Manager/Compiler flags it with a clear 'please confirm this with your doctor' note.

- The caregiver requests a change, for example 'add a question about my allergy history'; it is routed, re-verified and shown.
- The caregiver reviews and approves the result.
- Optional live moment: enter an emergency phrase to show the hard stop.

15. Team and 5-Day Plan

The team has **3 members: Ishaan, Mariam and Mozzam / Askeri**. The split below is a proposal that groups the work into three balanced workstreams; swap owners freely. Everyone helps on Day 1 to get vLLM running.

Owner (proposed)	Workstream	Scope
Ishaan	Infrastructure and orchestration	MI300X, ROCm and vLLM setup with one shared model; safety classifier hosting; Main Agent, care contract, routing, retry loop and human-review hand-off; emergency check
Mariam	Verification and review	Manager/Compiler verification logic (claim-to-source linking, confidence levels); chat Approve / Improve / Reject flow; evaluation harness and baseline comparison
Mozzam / Askeri	Workers and dashboard	The four Worker Agents (Intake, Document Classifier, Record Summarizer, Medication Checker) and drug data source; synthetic document set; care dashboard with live events and amd-smi metrics

Proposed schedule (adjust to your team)

Day	Milestone
1	GPU, ROCm and vLLM running with the chosen open model; event schema agreed; care-contract and output-format drafts; synthetic documents created
2	Worker Agents working individually; Main Agent dispatching in parallel; dashboard skeleton receiving events
3	Manager/Compiler verification and retry loop end to end; emergency check (trigger list plus classifier); amd-smi metrics live
4	Chat approve / improve / reject flow; full demo scenario with planted interaction; run 20-document baseline vs pipeline evaluation
5	Fix issues from evaluation, polish dashboard, rehearse demo, record backup video, finalize measured results, submit

16. Submission Checklist and Judging

Key dates from the event page: kickoff Monday 12 October 2026 (15:00 UTC); submissions close Sunday 18 October 2026 (15:00 UTC), when judging begins. Confirm times at kickoff.

Submission item	Notes
Project title, short and long descriptions	Select challenge: Track 2, Health and Wellbeing; add technology and category tags
Working prototype	Public GitHub repository, demo application platform and application URL
Cover image	One clear image for the project page

Submission item	Notes
Demo video and slide presentation	Show the case from first input to final result; explain the problem and intended user
Track 2 safety explanation	Explain handling of unsupported responses, sensitive information, uncertain results, high-risk decisions and human review

Judging criterion	How we address it
Application of Technology	One open model on MI300X via ROCm and vLLM, six agents sharing one GPU, small safety classifier, live amd-smi metrics
Business Value	Real caregiver problem, measured gains in time, accuracy and readability
Originality	Care contract plus claim-to-source verification and a human approval loop, all local
Presentation	Live dashboard, planted-error demo, honest baseline comparison

17. Risks and Mitigations

Risk	Mitigation
Keyword rules miss unusual emergency phrasing	Add a model-based classifier alongside the code trigger list; state the limitation openly; test with varied scripted phrasings
Model produces unsupported claims	Claim-level source linking and removal in the Manager/Compiler; measure unsupported-claim rate against baseline
Drug data source gaps or unavailable access	Default to 'cannot confirm' rather than guessing; use a small local trusted dataset for the demo if needed
Six agents on one model contend for the GPU	Single vLLM instance with continuous batching; monitor latency and memory on dashboard; keep prompts short
Retry loop adds latency	Hard limit of two retries; then low-confidence hand-off
Small team (3 people) and a tight timeline	Work to P0/P1/P2 priorities; share one prompt template across the four Workers; keep scope fixed; drop P2 items first
Perception that the tool gives medical advice	Fixed structure, persistent disclaimer, no diagnosis or treatment language, human approval required

18. Open Questions

- Which open model (and size) will be served on the MI300X, and what is the safety classifier?
- Which trusted drug data source will be used, and is it accessible offline for the demo?
- What document formats will the synthetic set use (PDF, scanned images, plain text)?
- Which readability formula will be used for the 8th-grade target?
- Will the approved summary be exportable in this version (FR-16)?
- Will the team join on-site (Rome, Milan or Imperia) or build fully online?

19. Future Scope

Hospital-server deployment pilots, EHR integration, additional languages, clinician-facing review queue, and larger-scale evaluation with expert-reviewed data. These are out of scope for this hackathon build.

PART 2

Project Workflow

End-to-end workflow: intake and safety screen, care contract, four Workers, verification and retry loop, human approval, live dashboard, evaluation and demo.

Pages 13-18 of 29

AMD ACT 3 HACKATHON / PROJECT WORKFLOW

Multi-Agent AI Care Coordination System

From medical document overload to a source-checked, patient-ready care summary.

LOCAL INFERENCE AMD Instinct MI300X	AGENT ORCHESTRATION Main Agent + 4 Workers + Compiler
RUNTIME ROCm + vLLM	HUMAN CONTROL Review before final approval

End-to-end workflow

The system screens the request, delegates document work, verifies every claim against its source, and keeps the patient or caregiver in control of final approval.

1	2	3	4	5	6
Request + documents	Safety screen	Main Agent care contract	Four Workers in parallel	Compiler verification	Patient review and approval

Safety gate
Current emergency or immediate danger in the request triggers a hard stop and urgent-help direction. No summary is generated.

Live observability
Agent events and MI300X metrics feed a dashboard showing activity, uncertainty, latency, tokens per second, and memory use.

Design principle
The system organizes and explains the information already present in the record. It does not diagnose or prescribe treatment. Unsupported or unresolved items are removed, flagged, or routed to a human reviewer.

TRACK 2 FIT | HEALTHCARE
This idea is a good fit for Track 2 (Healthcare). It applies local AI to help patients and caregivers organize medical records, prepare questions for clinicians, and surface uncertainty for human review.

TEAM ISHAAN | MARIAM | MOZZAM/ASKERI

01 / Intake, safety and agent orchestration

Every request starts with a user goal and a document set. The workflow turns that request into bounded tasks before agents begin extracting facts.

01

Patient request and document upload

A patient or caregiver enters a plain-language request, for example, “Help me prepare for my cardiology appointment tomorrow” or “Explain these discharge instructions in simple words.” They upload relevant discharge summaries, lab reports, prescriptions, specialist notes, or imaging reports.

02

Emergency safety screening

Before normal processing, a safety layer checks for a current emergency or immediate danger described in the request. A positive screen stops summary generation and directs the person to contact a clinician or their local emergency number. The check combines a fixed code-based trigger list with a smaller model-based classifier. The trigger list is a conservative safety net, not a guarantee that every emergency phrasing will be caught.

03

Main Agent: intent analysis and care contract

The Main Agent interprets the request, identifies the needed information, and writes a short shared care contract. It specifies the goal, relevant information, assigned Worker Agent, required output format, source-traceability rules, and uncertainty-handling requirements. This shared plan helps reduce overlap, missing work, and unsupported additions.

Four Worker Agents

Worker 1 | Intake Agent

Interprets the request in detail and identifies which documents are relevant to the patient's goal.

Worker 2 | Document Classifier Agent

Sorts and indexes the document set by type, including lab results, prescriptions, discharge notes, and imaging reports.

Worker 3 | Record Summarizer

Extracts relevant clinical facts from source documents, organizing history, results, and instructions without inventing facts.

Worker 4 | Medication Checker

Compares medication information against a trusted drug data source. Reports “cannot confirm” if a medicine or interaction cannot be verified.

Parallel execution with dependencies

The Worker Agents run in parallel where their tasks allow. Detailed extraction uses the organized document index, and every worker follows the same care contract. Missing evidence is treated as uncertainty rather than filled in with a guess.

02 / Local inference and source verification

The multi-agent workflow is backed by one local inference stack and a verification stage that makes the final output auditable.

AMD GPU and serving architecture

COMPONENT	ROLE IN THE SYSTEM
AMD Instinct MI300X	Provides 192 GB of HBM memory for the main open model, the smaller safety classifier, and runtime workloads.
ROCm	GPU software stack used to run the local inference workload.
vLLM	Serves one shared copy of the main model to all six agents: Main Agent, four Workers, and Manager/Compiler Agent.
Local deployment	AI inference runs on project-controlled hardware; patient data is not sent to third-party AI inference APIs. The design can be adapted to hospital-owned servers.

Manager/Compiler Agent: merge and verify

After the Workers return their results, the Manager/Compiler Agent merges them into one coherent care summary and checks each claim against the original source documents. Supported claims retain a source reference to the exact line or passage; untraceable claims are removed or flagged. Each section receives a confidence level.

1. Plain-language summary
A clear overview of the relevant information in the record.

2. Questions for the doctor
Questions suggested by the record and the patient's stated goal.

3. Medication and allergy checklist
Relevant medicines, allergies, discrepancies, and items to confirm.

4. Needs clinician review
Uncertain facts, unconfirmed interactions, and concerns requiring professional review.

Verification failure and retry loop

Check fails A claim cannot be traced or a required item is incomplete.	Route for correction Main Agent identifies the responsible Worker and sends the issue back.	Re-check The corrected output is verified again, with a maximum of two retries.	Escalate If unresolved, mark low-confidence and hand off to a human reviewer.

03 / Human approval and live monitoring

The output is not final when the model finishes writing. It becomes final only after the patient or caregiver explicitly approves it.

Patient or caregiver review loop

APPROVE
Accept the prepared summary. The system records explicit approval before marking it final.

IMPROVE
Ask for a change, such as adding a question about allergy history. The Main Agent routes the change to the correct Worker and repeats verification.

REJECT
Decline the prepared summary. It is not marked as approved or final.

Risk handling: request versus record history

SITUATION	SYSTEM RESPONSE
Current emergency or immediate danger described in the request	Hard stop: generate no care summary and direct the person to contact a clinician or their local emergency number.
Risky item mentioned only in medical history	Continue the document workflow, but flag the item in “Needs clinician review.”
Unverified medicine or possible interaction	State “cannot confirm” and route the item to a doctor or pharmacist; do not invent a match.
Claim cannot be traced to a source	Remove or flag it, retry within the limit, and escalate unresolved items to a human reviewer.

Live observability dashboard

Every agent emits simple live events. These events power a real-time dashboard that shows which agents are active, workflow progress, verification status, uncertainty flags, retry activity, and human-review items. GPU metrics are read live from `amd-smi` on the MI300X.

Agent state
Active stage and task status

Quality and safety
Uncertainty flags, source checks, retries

Performance
Latency and tokens per second

Hardware
GPU utilization and memory usage

Demo objective
Make the underlying work visible. Judges should be able to see real agent events, source-verification outcomes, retries, and GPU telemetry rather than only decorative status indicators.

04 / Evaluation, live demo and expected outcome

We will evaluate the pipeline on synthetic records, then demonstrate how it handles a deliberately planted uncertainty without guessing.

Evaluation plan

Run 20 synthetic documents through two paths: (1) a plain single-prompt baseline and (2) the full multi-agent pipeline. Compare the same types of outputs and report the measurements obtained. No real patient records are used.

METRIC	HOW IT IS MEASURED	PROJECT TARGET
Unsupported-claim rate	Count claims that are not supported by the source records, divided by claims reviewed.	Below 5%
Reading level	Estimate the readability of the patient-facing summary.	Approximately 8th grade
Preparation time	Measure end-to-end generation time and compare with a manually timed baseline.	Under 2 minutes
Planted-error detection	Count deliberately introduced errors identified by each approach.	Report observed counts

The targets above are goals, not established results. The submission will report actual measurements, including any target that is not met.

Live demo: from upload to approval

1	**Upload** A caregiver uploads a synthetic discharge summary, two lab reports, and a prescription list.
2	**Organize and delegate** The safety layer screens the request; the Main Agent creates the care contract; Workers process the records.
3	**Verify** The Compiler merges results and traces claims to source locations.
4	**Show uncertainty** A planted medication interaction cannot be confirmed; the Medication Checker flags it for a doctor or pharmacist.
5	**Review and approve** The caregiver reviews the summary, doctor questions, medication checklist, and review flags, then approves.

Team execution plan | 3 members

ISHAAN
Main Agent, care contract, ROCm/vLLM integration

MARIAM
Intake, classification, clinical fact extraction

MOZZAM/ASKERI
Medication checks, source verification, review/dashboard

Safety testing, end-to-end integration, and evaluation are shared across the team.

Final outcome

A transparent, safety-first, human-supervised care coordinator running locally on AMD GPUs. It turns large record sets into a clear, source-linked summary, preserves uncertainty, and does not provide diagnosis or treatment advice.

PART 3

Tech Stack and System Blueprint

Implementation map: specified vs proposed components (GPU and serving, orchestration, ingestion, medication data, dashboard, local data), safety and evaluation.

Pages 19-21 of 29

AMD DEVELOPER HACKATHON / ACT III

CARE COORDINATION AI

TECH STACK + SYSTEM BLUEPRINT

A local-first, multi-agent workflow that turns medical documents into a source-grounded care summary - with explicit safety gates and human approval.

TRACK 2 / HEALTH & WELLBEING

LOCAL INFERENCE

AMD + ROCm

MI300X | vLLM

CORE PLATFORM

Specified in the project brief

- AMD Instinct MI300X · 192 GB
- ROCm
- vLLM inference
- Open-weight model

WHY TRACK 2 FITS — Directly addresses medical-record summaries, appointment preparation, care navigation, and clearer discharge instructions - while making uncertainty and human review visible.

REQUEST TO VERIFIED, APPROVED SUMMARY

SOURCE-GROUNDED | HUMAN-APPROVED

1 REQUEST + DOCUMENTS

Plain-language goal and uploaded patient records

2 EMERGENCY SAFETY GATE

Fixed trigger rules + a small local classifier. A current emergency or danger signal means a hard stop and no summary.

3 MAIN AGENT + SHARED CARE CONTRACT

Defines required evidence, assigns each Worker, fixes the output schema, and coordinates bounded retries.

4 WORKER AGENTS RUN IN PARALLEL

INTAKE AGENT	CLASSIFIER	RECORD SUMMARIZER	MEDICATION CHECKER
Interpret request and select relevant records	Sort and index document pages by type	Extract source-supported clinical facts	Cross-check trusted drug data; flag unknowns

5 MANAGER / COMPILER AGENT

Merges worker outputs, traces claims to source page/line, assigns section confidence, and removes or flags unsupported claims. At most two retries, then human review.

6 PATIENT / CAREGIVER REVIEW — APPROVE / IMPROVE / REJECT

Nothing is final until explicit approval. The fixed output contains a plain-language summary, questions for the doctor, a medication/allergy checklist, and items that need clinician review.

IMPLEMENTATION MAP

STACK COMPONENTS, SAFETY & EVALUATION

Specified items come from the project brief; proposed items fill implementation gaps.

GPU + INFERENCE

SPECIFIED

AMD Instinct MI300X (192 GB), ROCm, and vLLM. Serve one open-weight instruction model from one local endpoint shared by all six agents. Pin the exact model checkpoint after quality, latency, and throughput tests.

AGENT ORCHESTRATION + API

PROPOSED

Proposed: Python 3.12, asyncio, Pydantic, and FastAPI. Use typed schemas and an explicit state machine for Main Agent orchestration, four parallel Workers, compiler verification, bounded retries, and approval state.

DOCUMENT INGESTION + PROVENANCE

PROPOSED

Proposed: PyMuPDF for text PDFs, plus local OCR for scanned pages. Preserve document ID, page, and line/span anchors alongside extracted facts so each final claim can point back to its evidence.

MEDICATION VERIFICATION

NEEDS SELECTION

Select an approved, versioned medication terminology source and a drug-interaction dataset with suitable coverage and licensing. Name matching alone is not interaction checking. Return “cannot confirm” when reliable evidence is absent.

CARE DASHBOARD + TELEMETRY

PROPOSED

Proposed: React + TypeScript, with WebSocket or server-sent events for agent activity, confidence flags, and review status. Read GPU utilization and memory through an amd-smi adapter; report latency and tokens/sec in the backend.

LOCAL DATA + DEPLOYMENT

PROPOSED

Keep documents, facts, citations, audit events, and approvals on a controlled host/network. Prototype option: encrypted local files plus PostgreSQL metadata. Package services in ROCm-compatible containers for AMD cloud or hospital servers.

SAFETY AND HUMAN CONTROL

- Emergency request: hard stop; create no care summary and direct the person to a clinician or local emergency number.
- Historical risk in a record: continue, but flag it for clinician review rather than treating it as a current emergency.
- Classifier limitation: code-based triggers are conservative; unusual phrasing can still be missed.
- Verification fails: retry the responsible Worker at most twice, then mark low confidence and route to a human.
- Human approval: patient/caregiver must approve before the result is final.
- Privacy: demonstrate with synthetic records; do not send patient documents to third-party AI APIs.

HACKATHON EVALUATION PLAN

Compare a plain single-prompt baseline with the full pipeline on the same 20 synthetic documents.

<5%	~8th grade	<2 min	Counted
unsupported-claim rate	reading level	preparation time	planted errors caught

Report measured results, not targets as achievements. Also track latency and unsupported claims removed or flagged.

Before demo: lock the model checkpoint, medication data source, and versions of proposed components.

TEAM

3 MEMBERS

ISHAAN | MARIAM | MOZZAM/ASKERI

PART 4

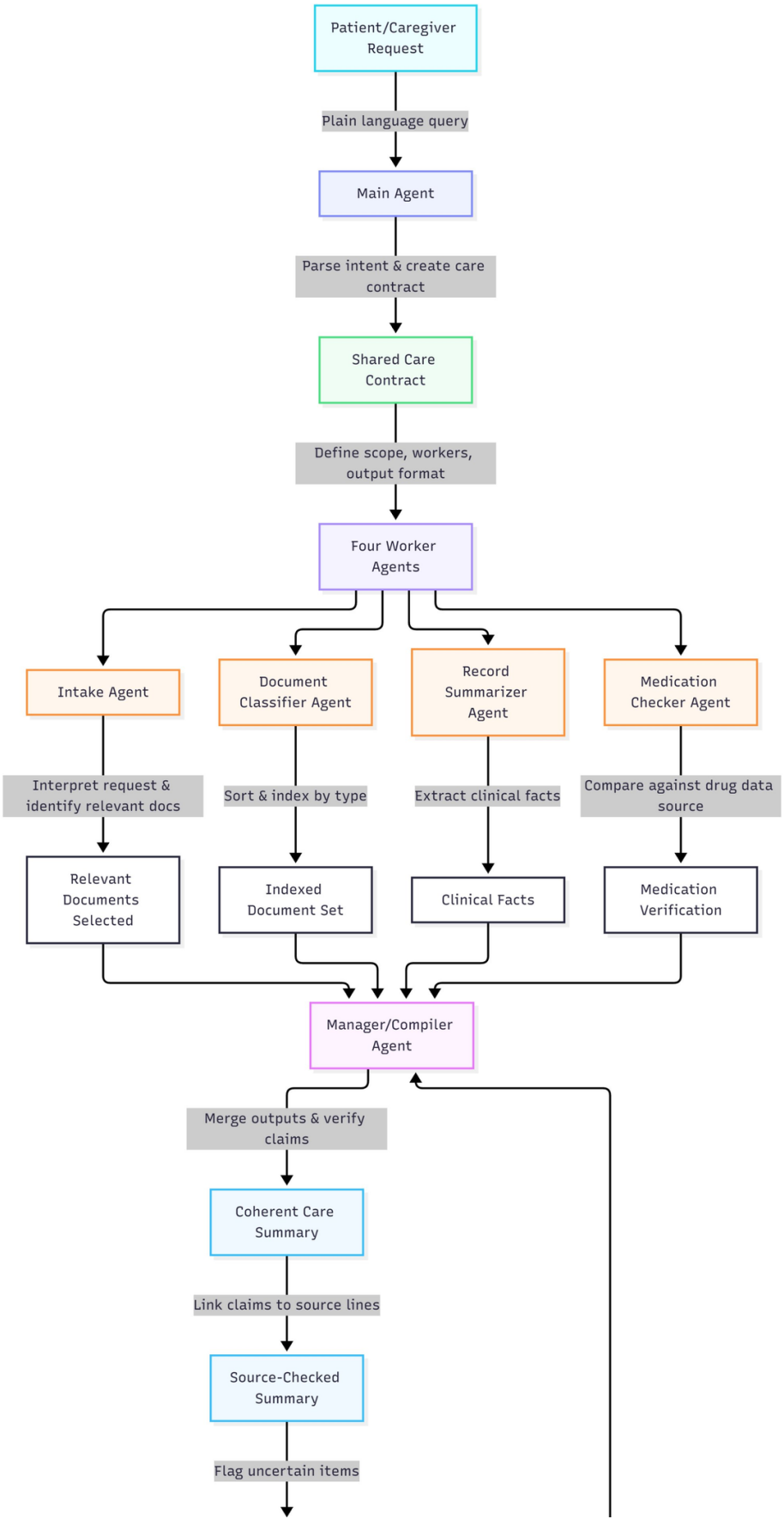
Workflow Flowchart

Visual flowchart of the full process from patient request to the live dashboard, shown on two pages (top half, then bottom half).

Pages 22-24 of 29

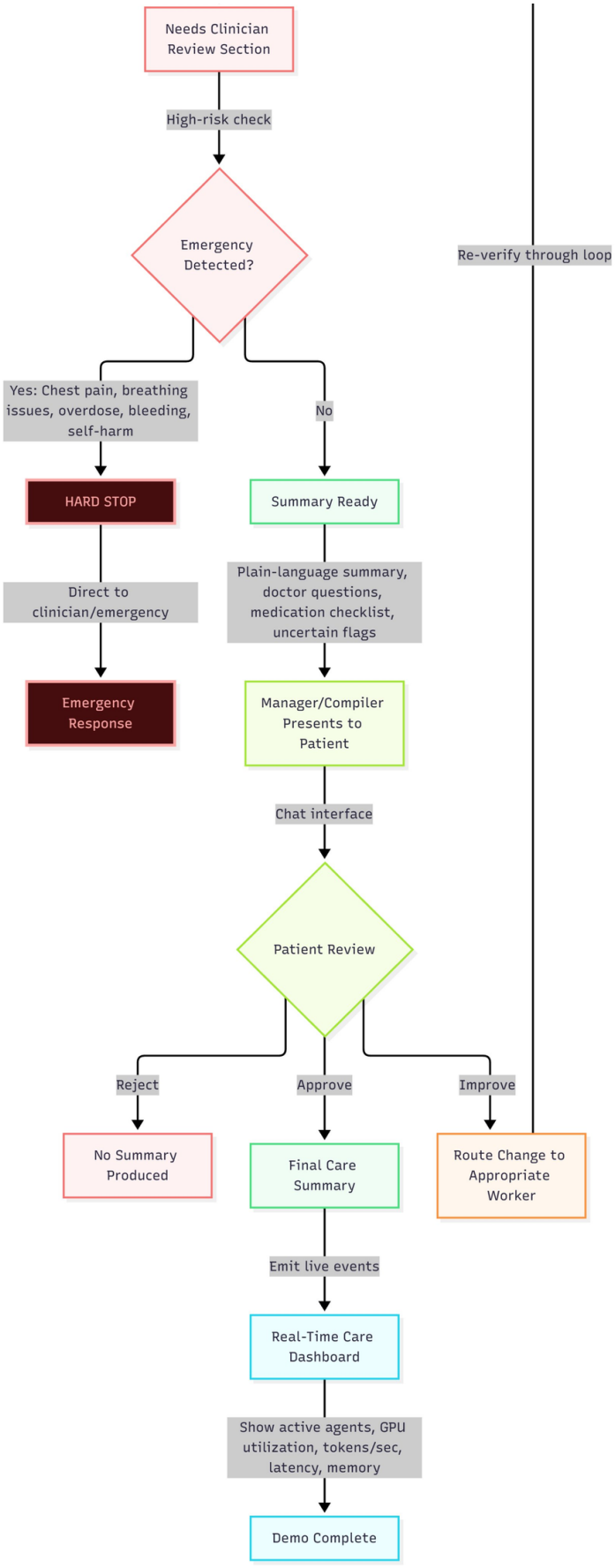
Part 4: Workflow Flowchart (1 of 2)

continues on next page



Part 4: Workflow Flowchart (2 of 2)

continued from previous page



PART 5

CarePilot AI Features

Project concept and system features: cancer-patient record organization, doctor-visit preparation, workflow, role of Python, safety and limitations.

Pages 25-29 of 29

CarePilot AI

Cancer Patient Medical Record Organization & Doctor-Visit Preparation

Project Concept and System Features

1. Project Overview

CarePilot AI is a proposed AI-assisted system designed to help organize medical records for cancer patients. Cancer patients may consult multiple doctors and collect reports from different appointments, hospitals, and stages of care. These documents can be scattered, making it time-consuming for a doctor to review the patient's available medical history.

CarePilot AI aims to bring supported documents into one organized workflow, extract and classify their information, arrange them chronologically, flag missing or conflicting details, and prepare a concise medical-history brief for a doctor to review. It is intended to support organization and communication—not to diagnose cancer, recommend treatment, or replace clinical judgment.

2. Main System Features

FEATURE 1

Medical Document Upload

An authorized user will be able to upload supported medical documents, such as laboratory reports, biopsy/pathology reports, CT or MRI reports, doctors' notes, and discharge summaries. The initial version may support only selected file types.

FEATURE 2

Document Reading and Text Extraction

Python-based processing will attempt to read supported files and extract text such as dates, test names, and reported findings. Scanned documents may require Optical Character Recognition (OCR). If a file cannot be read reliably, the system should mark it as unreadable or uncertain rather than inventing information.

FEATURE 3

Automatic Document Classification

The system will identify the likely category of each document—for example, a blood test as a laboratory report, a biopsy as a pathology report, a CT scan as a radiology report, or a doctor's note as a clinical note. This can reduce the need for manual sorting.

FEATURE 4

Chronological Medical Timeline

Documents will be arranged by their recorded dates so users can see which reports came earlier and later. For example: a blood test on January 5, a biopsy report on January 12, a CT scan on January 20, and a consultation note on January 28. If a date is missing or ambiguous, the system should flag it instead of guessing.

FEATURE 5

Medical History Summary

An AI component will prepare a concise summary of relevant information available in the documents. Depending on the records, this may include relevant medical history, report dates and findings, and information from previous consultations. Important summary points should, where feasible, link back to the original document or supporting passage so a clinician can verify them.

FEATURE 6

Missing or Conflicting Information Flags

The system will look for issues such as missing fields, unclear text, differing dates or details across documents, and summary claims without a clear source. For example, if two documents show different dates for the same test, the system should flag the discrepancy for review rather than decide which date is correct. It can only assess the documents available to it; it cannot know for certain that an unuploaded report exists.

FEATURE 7

Doctor-Visit Preparation

The system may generate a doctor-visit brief containing a concise medical history, a list of important reports, flagged issues, and suggested questions based on the available records. These questions are intended to support discussion with a clinician, not to provide diagnosis or treatment recommendations.

FEATURE 8

Multi-Agent AI Workflow

The proposed workflow can divide responsibilities among specialized components: (1) Main/Orchestrator Agent to coordinate tasks; (2) Document Intake component to manage uploaded files and metadata; (3) Classification Agent to identify document categories; (4) Summary Agent to draft the medical-history summary; (5) Verification component to compare summary claims with source documents; and (6) Manager/Review component to track errors and unresolved issues. Not every component needs a separate AI model—some tasks can be handled more reliably with ordinary Python functions and rules.

FEATURE 9

Patient Dashboard

The patient-facing interface may display uploaded documents, the medical timeline, the generated summary, and flagged issues. A login or sign-up screen can be included in the initial interface prototype, but it should not be described as secure, real authentication until a backend and authentication mechanism have been implemented.

FEATURE 10

System Monitoring Dashboard

A separate technical dashboard may show which workflow steps are running, agent/task status, processing errors and logs, and processing time. GPU utilization and memory metrics can be shown only when actual compatible hardware and monitoring are connected. Before that, any such figures must be clearly labelled as simulated demo data.

3. End-to-End Workflow

- **Upload:** The user uploads supported medical documents.
- **Extract:** Python tools attempt to extract readable text from the files.
- **Classify:** The system assigns a likely document category.
- **Organize:** Documents are arranged by date and category.

- **Summarize:** The AI component drafts a concise summary from the available records.
- **Verify:** Validation rules and a verification component compare key summary claims with source documents.
- **Flag:** Missing, unclear, unreadable, or conflicting information is marked for review.
- **Present:** The dashboard displays the doctor-visit brief and source references where available.

4. Role of Python

Python will be more than a way to call an AI model. It can provide much of the system's practical processing and control logic:

- **File handling:** Read and manage supported documents.
- **Data structures:** Use lists and dictionaries to organize extracted information.
- **Conditional logic:** Check whether text or dates are available and decide how to handle different cases.
- **Workflow coordination:** Run tasks in the intended order and pass structured results between components.
- **Validation:** Check required fields and data formats.
- **Comparison:** Compare relevant details across reports and identify potential discrepancies.
- **Error handling:** Handle corrupted files, extraction failures, and model errors safely.
- **Backend API:** Use a framework such as FastAPI later to connect the frontend with Python processing and the AI workflow.

5. How the Technology Components Fit Together

Component	Purpose
Python	Manages data, validation, and the overall workflow.
Large Language Model (LLM)	Helps interpret text and draft summaries from the available records.
Specialized agents/components	Handle distinct tasks within the workflow.
Verification logic	Checks generated claims against original source material where possible.
FastAPI	Can provide an API between the frontend and Python backend.
AMD GPU	Can accelerate supported model workloads when compatible hardware is available.
vLLM + ROCm	A possible software stack for serving compatible open-weight models on supported AMD hardware.

6. What Makes the Project Distinctive?

The project's goal is not simply to generate a medical summary. It is to build a structured, source-aware workflow for organizing multiple records and identifying issues that need human review.

- Bring multiple documents into one chronological view.
- Classify documents and preserve relevant details.
- Flag missing, unclear, or conflicting information.

- Make summary claims traceable to source documents where feasible.
- Coordinate specialized tasks through a Python-based workflow.
- Evaluate performance using measurable tests for classification, discrepancy detection, source support, and processing time.

A useful evaluation would compare the structured workflow with a simple single-prompt summarization baseline. This would help test whether the proposed approach improves organization, traceability, or issue detection rather than assuming that it does.

7. Safety, Privacy, and Limitations

Medical records are sensitive and must be handled with appropriate privacy and security protections.

- Use fictional or properly authorized sample records for demonstrations; do not use real patient records without permission and suitable safeguards.
- The system must not invent facts or silently resolve conflicting medical information.
- Uncertain, unreadable, or unsupported information should be clearly flagged.
- Clinicians remain responsible for medical interpretation and decisions.
- The system can only assess the records provided; it cannot confirm that all relevant records have been uploaded.

8. Proposed Features vs. Implemented Features

The items in this document describe the proposed design, not a claim that every feature has already been built. During a project presentation, distinguish clearly between (a) interface mock-ups, (b) features implemented in Python, (c) AI/model functionality that has been tested, and (d) future goals such as real GPU monitoring or deployment on AMD hardware.

9. Short Presentation Summary

> CarePilot AI is a proposed cancer-patient medical-record organization system. It will aim to read and classify supported reports, arrange them chronologically, prepare a concise medical-history summary, and flag missing or conflicting information for review. Python will manage document processing, validation, data organization, error handling, and workflow coordination, while AI components will help interpret medical text and draft summaries. The system is intended to help patients and clinicians access relevant records more efficiently; it will not diagnose disease or prescribe treatment.